Project Report

Package 4: Double McHarvard with Cheese Circular Shifts

Team Number	29
Team Name	Konafa
Package	Package 4

Name	Student ID	Tutorial
Youmna Osama Omar Mohamed	61-17731	8
Mariam Boulos Habashy	61-1773	8
Azza Khaled Darwish	61-1508	9
Farial Abdelrehim Ahmed	61-16271	9

Introduction

This report presents the design and implementation of a processor simulator for Package 4 (Double McHarvard with Cheese Circular Shifts). The simulator is written in C and accurately models a Harvard-architecture processor with a three-stage pipeline, twelve instructions, and a complete register file.

The project was implemented across six source files covering instruction memory, data memory, the register file, the ALU, and the three pipeline stages (Fetch, Decode, Execute).

Architecture Overview

Memory Architecture

Package 4 uses a Harvard architecture, meaning instruction and data memories are completely separate with independent address spaces and buses.

- **Instruction Memory** 1024 × 16 bits Word-addressable (16-bit word) Program instructions.
- **Data Memory** 2048 × 8 bits Byte-addressable Data values.

Register File

The processor has 66 registers in total:

- 64 General-Purpose Registers (R0–R63), each 8 bits wide.
- 1 Status Register (SREG), 8 bits — only bits 4:0 are used; bits 7:5 are always 0.
- 1 Program Counter (PC), 16 bits — special-purpose, not an 8-bit GPR.

SREG flag layout (bits 4:0): Bit 4 = C (Carry) | Bit 3 = V (Overflow) | Bit 2 = N (Negative) | Bit 1 = S (Sign) | Bit 0 = Z (Zero)

7	6	5	4	3	2	1	0
0	0	0	C	V	N	S	Z

Instruction Set Architecture

Instructions are 16 bits wide and come in two formats. All 12 opcodes are assigned values 0–11.

- **R-Format** [OPCODE 4][R1 6][R2 6] (ADD, SUB, MUL, AND, OR, JR).
- **I-Format** [OPCODE 4][R1 6][IMM 6] (LDI, BEQZ, SAL, SAR, LB, SB).

Instruction Reference Table

Name	Mnemonic	Type	Format	Operation
Add	ADD	R	ADD R1 R2	R1 = R1 + R2
Subtract	SUB	R	SUB R1 R2	R1 = R1 - R2
Multiply	MUL	R	MUL R1 R2	R1 = R1 * R2
Load Immediate	LDI	I	LDI R1 IMM	R1 = IMM
Branch if Equal Zero	BEQZ	I	BEQZ R1 IMM	IF(R1 == 0) { PC = PC+1+IMM }
And	AND	R	AND R1 R2	R1 = R1 & R2
Or	OR	R	OR R1 R2	R1 = R1 | R2
Jump Register	JR	R	JR R1 R2	PC = R1 || R2
Shift Arithmetic Left	SAL	I	SAL R1 IMM	R1 = R1 << IMM
Shift Arithmetic Right	SAR	I	SAR R1 IMM	R1 = R1 >> IMM
Load Byte	LB	I	LB R1 ADDRESS	R1 = MEM[ADDRESS]
Store Byte	SB	I	SB R1 ADDRESS	MEM[ADDRESS] = R1

SREG Flag Update Rules

- The **Carry flag (**C) is updated every **ADD** instruction.
- The **Overflow flag** (V) is updated every **ADD** and **SUB** instruction.
- The **Negative flag** (N) is updated every **ADD, SUB, MUL, AND, OR, SAL,** and **SAR** instruction.
- The **Sign flag** (S) is updated every **ADD** and **SUB** instruction.
- The **Zero flag** (Z) is updated every **ADD, SUB, MUL, AND, OR, SAL,** and **SAR** instruction.

Pipeline Design

Three-Stage Pipeline: IF → ID → EX

Clock cycle formula **cycles = 3 + (n − 1) × 1** where n = number of instructions.

Stage Descriptions

Instruction Fetch (IF) — fetch.c

- Reads inst_memory[PC] and stores the 16-bit word plus the current PC value into the IF/ID latch.
- Increments PC immediately after fetching.
- Returns false (latch set invalid) when PC ≥ program_size — the stopping condition.

Instruction Decode (ID) — decode.c

- Extracts ALL possible field interpretations from the 16-bit word.
- Sign-extends the 6-bit immediate for all I-format instructions except SAL/SAR (which use unsigned shift amounts).
- Reads register values from the register file, applying the 1-cycle forwarding path if a RAW hazard is detected.
- Fills the ID/EX latch.

Execute (EX) — execute.c

- Performs the operation specified by the opcode using the ALU (alu.c) or direct logic.
- Writes the result back to the destination register via apply_alu_result(), which also updates SREG.
- For LB: reads DataMem[ADDRESS] and writes it to R1.
- For SB: writes R1 to DataMem[ADDRESS]; no register write-back.
- For BEQZ/JR: sets branch_taken = true and branch_target = new PC if condition is met.
- Updates last_written_reg / last_written_value for the forwarding path.
- Clears the ID/EX latch (valid = false) after consuming it.

Example For The Pipeline:

Package 4 Pipeline			
	Instruction Fetch (IF)	**Instruction Decode (ID)**	**Execute (EX)**
Cycle 1	Instruction 1		
Cycle 2	Instruction 2	Instruction 1	
Cycle 3	Instruction 3	Instruction 2	Instruction 1
Cycle 4	Instruction 4	Instruction 3	Instruction 2
Cycle 5	Instruction 5	Instruction 4	Instruction 3
Cycle 6	Instruction 6	Instruction 5	Instruction 4
Cycle 7	Instruction 7	Instruction 6	Instruction 5
Cycle 8		Instruction 7	Instruction 6
Cycle 9			Instruction 7

Files and Sample Results

File	Responsibilite
main.c	Top-level simulation loop. Initialises all components, loads the program, drives the cycle loop (EX → ID → IF order), handles the branch/jump flush, checks the termination condition, and prints final state.
fetch.c	Instruction Fetch stage. Reads inst_memory[PC], fills the IF/ID latch, increments PC. Provides flush_IF_ID() to clear the latch on a branch.
decode.c	Instruction Decode stage. Extracts all field formats from the raw 16-bit word, sign-extends immediates, reads registers with forwarding, fills the ID/EX latch. Provides flush_ID_EX().
execute.c	Execute stage. Dispatches to the appropriate ALU function or handles memory/branch/jump directly. Updates SREG via apply_alu_result(), sets forwarding state, signals branch_taken/branch_target.
alu.c	All arithmetic and logic operations: alu_add, alu_sub, alu_mul, alu_and, alu_or, alu_sal, alu_sar. Each returns an ALUResult struct carrying the result and flag updates (−1 = do not update).
alu.h	Shared header: opcode constants (OP_ADD … OP_SB), flag bit positions (C/V/N/S/Z), ALUResult struct, and pipeline latch structs (IF_ID_Reg, ID_EX_Reg).
registers.c	64×8-bit GPR array, 16-bit PC, 8-bit SREG. Provides init_registers(), read_gpr(), write_gpr(), apply_alu_result() (flag update + register write), and print_all_registers().
InstructionMem.c	1024×16-bit instruction memory. Provides load_program() (text-file parser → encodes each instruction as a 16-bit word), read_instruction_memory(), and print_instruction_memory().
DataMem.c	2048×8-bit data memory. Provides init_data_memory(), read_data_memory(), write_data_memory(), and print_data_memory() (non-zero locations only).

Code Snippets

alu.c

alu.c — make_result() initialiser + alu_add() with full flag computation

```
static ALUResult make_result(void) {
    ALUResult r;
    r.result = 0; r.carry = -1; r.overflow = -1;
    r.negative = -1; r.zero = -1; r.sign = -1;
    return r;
}

ALUResult alu_add(uint8_t a, uint8_t b) {
    ALUResult res = make_result();
    uint16_t ur  = (uint16_t)a + (uint16_t)b;
    res.result   = (uint8_t)(ur & 0xFF);
    res.carry    = ((ur >> 8) & 1) ? 1 : 0;
    int sa = (a & 0x80)?1:0, sb=(b & 0x80)?1:0;
    int sr = (res.result & 0x80) ? 1 : 0;
    res.overflow = ((sa==sb)&&(sr!=sa)) ? 1 : 0;
    res.negative = (res.result & 0x80) ? 1 : 0;
    res.sign = res.negative ^ res.overflow;
    res.zero = (res.result == 0) ? 1 : 0;
    printf("     [ALU] ADD: %d + %d = %d\n",
           (int8_t)a, (int8_t)b, (int8_t)res.result);
    printf("     [ALU] C=%d  V=%d  N=%d  S=%d  Z=%d\n",
           res.carry,res.overflow,res.negative,res.sign,res.zero);
    return res;
}
```

fetch.c

fetch.c — fetchStage() reads instruction memory and flush_IF_ID()

```
bool fetchStage(void) {
    int prog_size = get_program_size();
    if ((int)PC >= prog_size) {
        IF_ID.valid = false;
        IF_ID.instruction = 0;
        IF_ID.fetchedPC   = PC;
        return false;
    }
    uint16_t instr    = read_instruction_memory(PC);
    IF_ID.instruction = instr;
    IF_ID.fetchedPC   = PC;
    IF_ID.valid       = true;
    int opcode = (instr >> 12) & 0xF;
    int r1     = (instr >>  6) & 0x3F;
    int r2_imm =  instr        & 0x3F;
    printf("  [IF] Fetched: PC=%-4d  0x%04X  (opcode=%d  R1=%d  R2/IMM=%d)\n",
           (int)PC, instr, opcode, r1, r2_imm);
    PC++;
    return true;
}

void flush_IF_ID(void) {
    IF_ID.valid       = false;
    IF_ID.instruction = 0;
    IF_ID.fetchedPC   = 0;
    printf("  [IF] -- flushed (IF/ID cleared due to branch/jump) --\n");
```

```
}
```

decode.c

decode.c — sign_extend_6(), full decodeStage() with forwarding, flush_ID_EX()

```
static int sign_extend_6(int raw6) {
    if (raw6 & 0x20)
        return raw6 | (~0x3F);
    return raw6;
}

void decodeStage(void) {
    if (!IF_ID.valid) {
        ID_EX.valid = false;
        printf("  [ID] -- no instruction (latch empty) --\n");
        return;
    }
    uint16_t instr = IF_ID.instruction;
    int opcode = (instr >> 12) & 0xF;
    int r1     = (instr >>  6) & 0x3F;
    int r2_raw =  instr        & 0x3F;
    int imm_raw=  instr        & 0x3F;
    bool is_shift = (opcode==OP_SAL || opcode==OP_SAR);
    int imm = is_shift ? (imm_raw & 0x3F) : sign_extend_6(imm_raw);
    uint8_t valR1, valR2;
    if (last_write_valid && last_written_reg == r1)
        valR1 = last_written_value;
    else
        valR1 = read_gpr(r1);
    if (last_write_valid && last_written_reg == r2_raw)
        valR2 = last_written_value;
    else
        valR2 = read_gpr(r2_raw);
    ID_EX.opcode=opcode; ID_EX.r1=r1; ID_EX.r2=r2_raw;
    ID_EX.imm=imm; ID_EX.valR1=valR1; ID_EX.valR2=valR2;
    ID_EX.instrPC=IF_ID.fetchedPC; ID_EX.valid=true;
}

void flush_ID_EX(void) {
    ID_EX.valid=false; ID_EX.opcode=-1;
    printf("  [ID] -- flushed (ID/EX cleared due to branch/jump) --\n");
}
```

execute.c

execute.c — BEQZ branch handling

```
case OP_BEQZ:
    if (valR1 == 0) {
        branch_target = (uint16_t)(instrPC + 1 + imm);
        branch_taken  = true;
        PC = branch_target;
    }
    last_write_valid = false;
    break;
```

registers.c

registers.c — apply_alu_result() updates SREG

```
void apply_alu_result(ALUResult res, int dest_reg,
                      const char *stage) {
```

```
    if (res.carry    >= 0) SET_FLAG(C_FLAG, res.carry);
    if (res.overflow >= 0) SET_FLAG(V_FLAG, res.overflow);
    if (res.negative >= 0) SET_FLAG(N_FLAG, res.negative);
    if (res.sign     >= 0) SET_FLAG(S_FLAG, res.sign);
    if (res.zero     >= 0) SET_FLAG(Z_FLAG, res.zero);
    SREG &= 0x1F;          /* keep only bits 4:0 */
    if (dest_reg >= 0)
        write_gpr(dest_reg, res.result, stage);
}
```

InstructionMem.c

InstructionMem.c — print_instruction_memory()

```
void print_instruction_memory(void) {
    printf("\n========== INSTRUCTION MEMORY ==========\n");
    if (program_size == 0) {
        printf("  (empty)\n");
    } else {
        for (int i = 0; i < program_size; i++) {
            uint16_t w = inst_memory[i];
            int op = (w >> 12) & 0xF;
            int r1 = (w >>  6) & 0x3F;
            int r2 =  w        & 0x3F;
            printf("InstrMem[%4d] = 0x%04X  (op=%2d r1=%2d r2/imm=%2d)\n",
                   i, w, op, r1, r2);
        }
    }
    printf("=========================================\n");
}
```

DataMem.c

DataMem.c — all four functions: init, read, write, print

```
void init_data_memory(void) {
    memset(data_memory, 0, sizeof(data_memory));
}

uint8_t read_data_memory(uint16_t address) {
    if (address >= DATA_MEM_SIZE) {
        fprintf(stderr, "[DataMem] ERROR: read address %u out of range\n",
                (unsigned)address);
        return 0;
    }
    return data_memory[address];
}

void write_data_memory(uint16_t address, uint8_t value,
                       const char *stage) {
    if (address >= DATA_MEM_SIZE) {
        fprintf(stderr, "[DataMem] ERROR: write address %u out of range\n",
                (unsigned)address);
        return;
    }
    uint8_t old = data_memory[address];
    data_memory[address] = value;
    printf("     [%s] DataMem[%u] updated: %d (0x%02X) -> %d (0x%02X)\n",
           stage,(unsigned)address,(int8_t)old,old,(int8_t)value,value);
}

void print_data_memory(void) {
```

```
    printf("\n========== DATA MEMORY (non-zero locations) ==========\n");
    int printed = 0;
    for (int i = 0; i < DATA_MEM_SIZE; i++) {
        if (data_memory[i] != 0) {
            printf("DataMem[%4d] = %4d (0x%02X)\n",
                   i, (int8_t)data_memory[i], data_memory[i]);
            printed++;
        }
    }
    if (printed == 0)
        printf("  (all zeros)\n");
    printf("=====================================================\n");
}
```

main.c

main.c — pipeline loop with branch flush

```
while (1) {
    cycle++;
    printf("\n============ Clock Cycle %-4d ============\n", cycle);
    executeStage();
    if (branch_taken) {
        flush_ID_EX();
        flush_IF_ID();
        branch_taken = false;
        fetchStage();
    } else {
        decodeStage();
        fetchStage();
    }
    if (!IF_ID.valid && !ID_EX.valid) {
        int formula_cycles = 3 + (n - 1);
        while (cycle < formula_cycles) {
            cycle++;
            printf("\n============ Clock Cycle %-4d ============\n", cycle);
            printf("  [IF] -- no instruction --\n");
            printf("  [ID] -- no instruction (latch empty) --\n");
            printf("  [EX] -- no instruction (latch empty) --\n");
        }
        printf("\n[MAIN] Pipeline drained - simulation complete.\n");
        break;
    }
    if (cycle > 100000) {
        printf("\n[MAIN] Max cycle limit reached - possible infinite loop.\n");
        break;
    }
}
```

Sample Simulation Output

Below is a representative extract of the console output for cycles 1–4 and the two branch events, illustrating the per-cycle stage printout and forwarding messages.

Pipeline Startup (Cycles 1–4)

```
  [IF] Fetched: PC=0    0x3045  (opcode=3  R1=1  R2/IMM=5)

============ Clock Cycle 1 ============
  [EX] -- no instruction (latch empty) --
  [ID] Decode: PC=0    0x3045
       opcode=3  R1=1  R2=5  IMM=5
       value[R1]=0   value[R5]=0
  [IF] Fetched: PC=1    0x3083  (opcode=3  R1=2  R2/IMM=3)

============ Clock Cycle 2 ============
  [EX] Execute: opcode=3  R1=1  R2=5  IMM=5
     [EX] R1 updated: 0 (0x00) -> 5 (0x05)
     [EX] LDI R1 = 5
  [ID] Decode: PC=1    0x3083
       opcode=3  R1=2  R2=3  IMM=3
  [IF] Fetched: PC=2    0x0042  (opcode=0  R1=1  R2/IMM=2)

============ Clock Cycle 3 ============
  [EX] Execute: opcode=3  R1=2  R2=3  IMM=3
     [EX] R2 updated: 0 (0x00) -> 3 (0x03)
  [ID] Forwarding: R2 = 3 (from EX)
  [ID] Decode: PC=2  ADD R1 R2  value[R1]=5  value[R2]=3
  [IF] Fetched: PC=3    0x1042  (opcode=1  R1=1  R2/IMM=2)

============ Clock Cycle 4 ============
  [EX] Execute: opcode=0  R1=1  R2=2  (ADD)
     [ALU] ADD: 5 + 3 = 8
     [ALU] C=0  V=0  N=0  S=0  Z=0
     [EX] R1 updated: 5 (0x05) -> 8 (0x08)
     [EX] SREG = 0x00  (C=0 V=0 N=0 S=0 Z=0)
```

BEQZ Taken (Cycle 23)

```
============ Clock Cycle 23 ============
  [EX] BEQZ R10 (0) IMM=1  instrPC=21
  [EX] Branch TAKEN -> new PC = 23
  [ID] -- flushed (ID/EX cleared due to branch/jump) --
  [IF] -- flushed (IF/ID cleared due to branch/jump) --
  [IF] Fetched: PC=23   0x3314  (opcode=3  R1=12  R2/IMM=20)
```

JR Taken (Cycle 28)

```
============ Clock Cycle 28 ============
  [EX] JR: R13=0x00  R14=0x1D  -> PC = 29
  [ID] -- flushed (ID/EX cleared due to branch/jump) --
  [IF] -- flushed (IF/ID cleared due to branch/jump) --
  [IF] Fetched: PC=29   0x347F  (opcode=3  R1=17  R2/IMM=63)
```

Carry/Overflow Test (Cycle 32)

```
============ Clock Cycle 32 ============
  [EX] Execute: opcode=0  ADD R17 R18
     [ALU] ADD: -1 + -1 = -2
     [ALU] C=1  V=0  N=1  S=1  Z=0
     [EX] R17 updated: -1 (0xFF) -> -2 (0xFE)
     [EX] SREG = 0x16  (C=1 V=0 N=1 S=1 Z=0)
```

Constraints Summary

Parameter	Value	Reason
Signed immediate (LDI/BEQZ/LB/SB)	−32 to +31	6-bit signed field
Shift amount (SAL/SAR)	0 to 63	6-bit unsigned field
Register index	0 to 63	6-bit field
Instruction memory	0 to 1023 addresses	10-bit address space
Data memory (direct)	0 to 31 via LB/SB IMM	6-bit signed IMM max = 31
Max program length	1024 instructions	Instruction memory size

Conclusion

The Package 4 processor simulator fully implements the Double McHarvard architecture as specified. All 12 ISA instructions are correctly encoded, decoded, and executed. The 3-stage pipeline (IF → ID → EX) runs with 1-cycle data forwarding to resolve RAW hazards and a flush-and-fetch mechanism to handle control hazards from branches and jumps.

The implementation is generic: any syntactically valid Package 4 assembly program can be loaded and simulated without modifying the C source. The test program provided covers all instructions, both branch outcomes.